```
import time
import heapq
import math
from collections import deque

UNWEIGHTED = {
    "A": ["B", "D"],
    "B": ["A", "C", "D"],
    "C": ["B", "E", "G"],
    "D": ["A", "B", "E"],
    "E": ["C", "D", "F", "H"],
    "F": ["E", "H"],
    "G": ["C", "H"],
    "H": ["E", "F", "G"]
}

WEIGHTED = {
    "A": {"B": 4, "D": 2},
    "B": {"A": 4, "C": 5, "D": 1},
    "C": {"B": 5, "E": 3, "G": 6},
    "D": {"A": 2, "B": 1, "E": 7},
    "E": {"C": 3, "D": 7, "F": 2, "H": 5},
    "F": {"E": 2, "H": 1},
    "G": {"C": 6, "H": 2},
    "H": {"E": 5, "F": 1, "G": 2}
}

COORDS = {
    "A": (0, 0),
    "B": (2, 1),
    "C": (4, 1),
    "D": (2, 3),
    "E": (5, 4),
    "F": (7, 3),
    "G": (6, 0),
    "H": (8, 1)
}

START = "A"
GOAL = "H"


def heuristic(node, goal):
    x1, y1 = COORDS[node]
    x2, y2 = COORDS[goal]
    return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)


def path_cost(path):
    if not path or len(path) < 2:
        return 0
    return sum(WEIGHTED[a][b] for a, b in zip(path, path[1:]))


def make_result(name, path, nodes, start_time):
    found = path is not None
    return {
        "name": name,
        "found": found,
        "path": path if found else [],
        "cost": path_cost(path) if found else "N/A",
        "length": len(path) - 1 if found else "N/A",
        "nodes": nodes,
        "time": time.perf_counter() - start_time
    }


def bfs(start, goal):
    start_time = time.perf_counter()
    queue = deque([[start]])
    visited = {start}
    nodes = 0

    while queue:
        path = queue.popleft()
```

```
        node = path[-1]
        nodes += 1

        if node == goal:
            return make_result("BFS", path, nodes, start_time)

        for neighbor in UNWEIGHTED[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])

    return make_result("BFS", None, nodes, start_time)


def dfs(start, goal):
    start_time = time.perf_counter()
    stack = [[start]]
    visited = set()
    nodes = 0

    while stack:
        path = stack.pop()
        node = path[-1]

        if node in visited:
            continue

        visited.add(node)
        nodes += 1

        if node == goal:
            return make_result("DFS", path, nodes, start_time)

        for neighbor in reversed(UNWEIGHTED[node]):
            if neighbor not in visited:
                stack.append(path + [neighbor])

    return make_result("DFS", None, nodes, start_time)


def ucs(start, goal):
    start_time = time.perf_counter()
    queue = [(0, 0, [start])]
    best = {start: 0}
    counter = 0
    nodes = 0

    while queue:
        cost, _, path = heapq.heappop(queue)
        node = path[-1]
        nodes += 1

        if node == goal:
            return make_result("UCS", path, nodes, start_time)

        for neighbor, weight in WEIGHTED[node].items():
            new_cost = cost + weight

            if neighbor not in best or new_cost < best[neighbor]:
                best[neighbor] = new_cost
                counter += 1
                heapq.heappush(
                    queue,
                    (new_cost, counter, path + [neighbor])
                )

    return make_result("UCS", None, nodes, start_time)


def dls(start, goal, limit):
    start_time = time.perf_counter()
    nodes = [0]

    def search(path, depth):
        nodes[0] += 1
        node = path[-1]

        if node == goal:
```

```
            return path

        if depth == limit:
            return None

        for neighbor in UNWEIGHTED[node]:
            if neighbor not in path:
                result = search(path + [neighbor], depth + 1)
                if result:
                    return result

        return None

    path = search([start], 0)
    return make_result(f"DLS({limit})", path, nodes[0], start_time)


def ids(start, goal, max_depth):
    start_time = time.perf_counter()
    total_nodes = 0

    def search(path, depth, limit):
        nonlocal total_nodes
        total_nodes += 1

        node = path[-1]

        if node == goal:
            return path

        if depth == limit:
            return None

        for neighbor in UNWEIGHTED[node]:
            if neighbor not in path:
                result = search(path + [neighbor], depth + 1, limit)
                if result:
                    return result

        return None

    for limit in range(max_depth + 1):
        path = search([start], 0, limit)

        if path:
            return make_result("IDS", path, total_nodes, start_time)

    return make_result("IDS", None, total_nodes, start_time)


def bidirectional(start, goal):
    start_time = time.perf_counter()

    if start == goal:
        return make_result("Bidirectional", [start], 1, start_time)

    forward_queue = deque([start])
    backward_queue = deque([goal])

    forward_parent = {start: None}
    backward_parent = {goal: None}

    while forward_queue and backward_queue:

        current = forward_queue.popleft()

        for neighbor in UNWEIGHTED[current]:
            if neighbor not in forward_parent:
                forward_parent[neighbor] = current
                forward_queue.append(neighbor)

                if neighbor in backward_parent:
                    path1 = []
                    node = neighbor

                    while node is not None:
                        path1.append(node)
```

```
                        node = forward_parent[node]

                    path1.reverse()

                    path2 = []
                    node = backward_parent[neighbor]

                    while node is not None:
                        path2.append(node)
                        node = backward_parent[node]

                    return make_result(
                        "Bidirectional",
                        path1 + path2,
                        len(forward_parent) + len(backward_parent),
                        start_time
                    )

        current = backward_queue.popleft()

        for neighbor in UNWEIGHTED[current]:
            if neighbor not in backward_parent:
                backward_parent[neighbor] = current
                backward_queue.append(neighbor)

                if neighbor in forward_parent:
                    path1 = []
                    node = neighbor

                    while node is not None:
                        path1.append(node)
                        node = forward_parent[node]

                    path1.reverse()

                    path2 = []
                    node = backward_parent[neighbor]

                    while node is not None:
                        path2.append(node)
                        node = backward_parent[node]

                    return make_result(
                        "Bidirectional",
                        path1 + path2,
                        len(forward_parent) + len(backward_parent),
                        start_time
                    )

    return make_result("Bidirectional", None, 0, start_time)


def greedy(start, goal):
    start_time = time.perf_counter()
    queue = [(heuristic(start, goal), 0, [start])]
    visited = {start}
    counter = 0
    nodes = 0

    while queue:
        _, _, path = heapq.heappop(queue)
        node = path[-1]
        nodes += 1

        if node == goal:
            return make_result("Greedy", path, nodes, start_time)

        for neighbor in WEIGHTED[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                counter += 1
                heapq.heappush(
                    queue,
                    (heuristic(neighbor, goal),
                     counter,
                     path + [neighbor])
                )
```

```
    return make_result("Greedy", None, nodes, start_time)


def a_star(start, goal):
    start_time = time.perf_counter()
    queue = [(heuristic(start, goal), 0, 0, [start])]
    best = {start: 0}
    counter = 0
    nodes = 0

    while queue:
        f, _, cost, path = heapq.heappop(queue)
        node = path[-1]
        nodes += 1

        if node == goal:
            return make_result("A*", path, nodes, start_time)

        for neighbor, weight in WEIGHTED[node].items():
            new_cost = cost + weight

            if neighbor not in best or new_cost < best[neighbor]:
                best[neighbor] = new_cost
                counter += 1
                new_f = new_cost + heuristic(neighbor, goal)

                heapq.heappush(
                    queue,
                    (new_f, counter, new_cost, path + [neighbor])
                )

    return make_result("A*", None, nodes, start_time)


def objective(x):
    return -(x - 3) ** 2 + 10


def hill_climbing(start_x):
    start_time = time.perf_counter()
    x = start_x
    iterations = 0

    while True:
        neighbors = [x - 1, x + 1]
        best = max(neighbors, key=objective)
        iterations += 1

        if objective(best) > objective(x):
            x = best
        else:
            break

    return {
        "initial": start_x,
        "final": x,
        "value": objective(x),
        "iterations": iterations,
        "time": time.perf_counter() - start_time
    }


def n_queens(n):
    start_time = time.perf_counter()
    board = [-1] * n
    assignments = 0
    backtracks = 0

    def safe(row, col):
        for r in range(row):
            if board[r] == col:
                return False

            if abs(board[r] - col) == abs(r - row):
                return False

        return True
```

```
    def solve(row):
        nonlocal assignments, backtracks

        if row == n:
            return True

        for col in range(n):
            assignments += 1

            if safe(row, col):
                board[row] = col

                if solve(row + 1):
                    return True

                board[row] = -1
                backtracks += 1

        return False

    found = solve(0)

    return {
        "n": n,
        "found": found,
        "solution": board if found else None,
        "assignments": assignments,
        "backtracks": backtracks,
        "time": time.perf_counter() - start_time
    }


def print_results(results):
    print()
    print(
        f"{'Algorithm':<16}"
        f"{'Found':<8}"
        f"{'Time':<12}"
        f"{'Nodes':<8}"
        f"{'Length':<9}"
        f"{'Cost':<8}"
    )

    print("-" * 61)

    for result in results:
        print(
            f"{result['name']:<16}"
            f"{str(result['found']):<8}"
            f"{result['time']:<12.6f}"
            f"{result['nodes']:<8}"
            f"{str(result['length']):<9}"
            f"{str(result['cost']):<8}"
        )

        if result["found"]:
            print("Path:", " -> ".join(result["path"]))


def run_uninformed():
    return [
        bfs(START, GOAL),
        dfs(START, GOAL),
        ucs(START, GOAL),
        dls(START, GOAL, 3),
        ids(START, GOAL, 6),
        bidirectional(START, GOAL)
    ]


def run_informed():
    return [
        greedy(START, GOAL),
        a_star(START, GOAL)
    ]
```

```
def menu():
    while True:
        print("\n===== AI SEARCH ALGORITHMS =====")
        print("1. Uninformed Search")
        print("2. Informed Search")
        print("3. Local Search")
        print("4. Constraint Satisfaction")
        print("5. Performance Comparison")
        print("6. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            print_results(run_uninformed())

        elif choice == "2":
            print_results(run_informed())

        elif choice == "3":
            for start in [-10, 0, 10]:
                result = hill_climbing(start)
                print(
                    f"Start: {start}, "
                    f"Final: {result['final']}, "
                    f"Value: {result['value']}, "
                    f"Iterations: {result['iterations']}, "
                    f"Time: {result['time']:.6f}s"
                )

        elif choice == "4":
            for n in [4, 8]:
                result = n_queens(n)
                print(
                    f"N={n}, "
                    f"Found={result['found']}, "
                    f"Assignments={result['assignments']}, "
                    f"Backtracks={result['backtracks']}, "
                    f"Time={result['time']:.6f}s"
                )

        elif choice == "5":
            results = run_uninformed() + run_informed()
            print_results(results)

        elif choice == "6":
            print("Program ended.")
            break

        else:
            print("Invalid choice.")


if __name__ == "__main__":
    menu()
```

```
===== AI SEARCH ALGORITHMS =====
1. Uninformed Search
2. Informed Search
3. Local Search
4. Constraint Satisfaction
5. Performance Comparison
6. Exit
Enter choice: 5

Algorithm       Found   Time        Nodes   Length   Cost
------------------------------------------------------------
BFS             True    0.000028    8       3        14
Path: A -> D -> E -> H
DFS             True    0.000018    7       5        15
Path: A -> B -> C -> E -> F -> H
UCS             True    0.000022    8       4        12
Path: A -> D -> E -> F -> H
DLS(3)          True    0.000018    14      3        14
Path: A -> D -> E -> H
IDS             True    0.000018    25      3        14
Path: A -> D -> E -> H
Bidirectional   True    0.000012    9       4        17
```

```
Path: A -> B -> C -> E -> H
Greedy          True    0.000021    5       4       17
Path: A -> B -> C -> G -> H
A*              True    0.000021    8       4       12
Path: A -> D -> E -> F -> H

===== AI SEARCH ALGORITHMS =====
1. Uninformed Search
2. Informed Search
3. Local Search
4. Constraint Satisfaction
5. Performance Comparison
6. Exit
Enter choice:
```